서브그래프는 어디에 끼어 있나

TokenZswap · `code/src/AMM.sol` → `code/subgraph/` → `frontend/src/`

> **한 줄 요약.** 솔리디티 컨트랙트가 **이벤트**를 남기면, 서브그래프(`amm.ts`)가 그 이벤트를 받아서 검색하기 좋은 **DB**로 정리해 둬요. 프론트엔드는 그 DB를 **GraphQL**로 조회해요. 서브그래프는 컨트랙트와 프론트엔드 사이에 끼는 **세 번째 프로그램**이에요.

1 전체 흐름

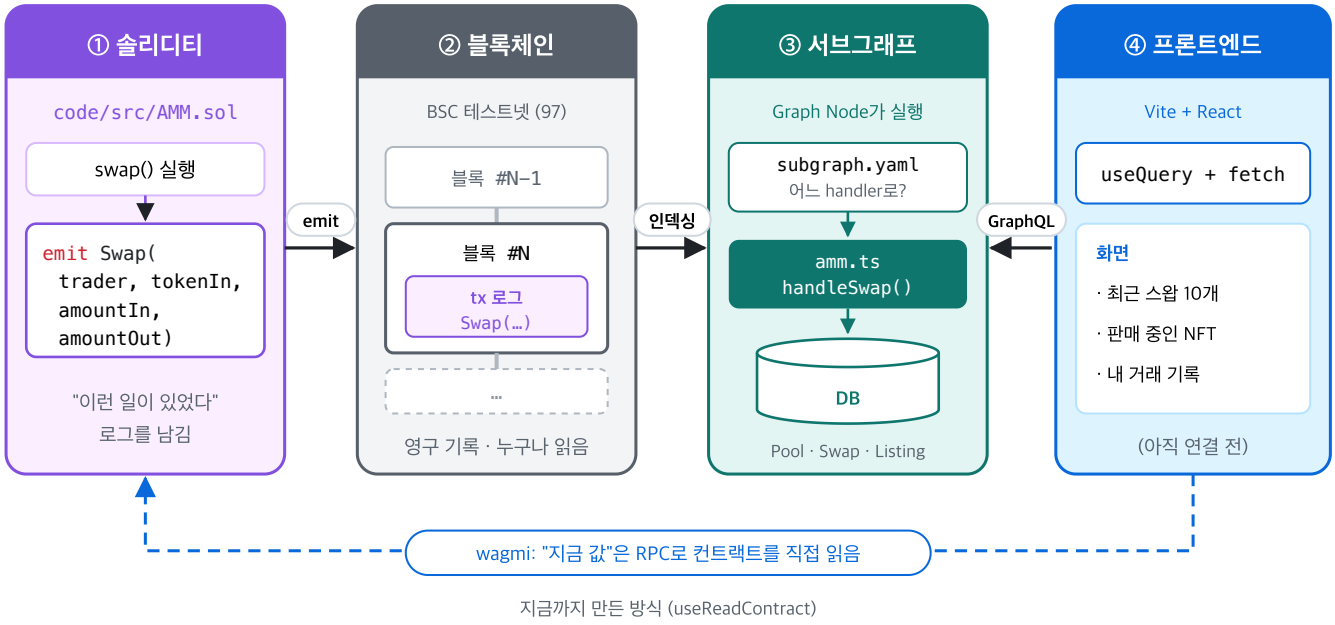


그림 1. 위쪽 경로(①→②→③→④)가 서브그래프. 아래 점선이 지금까지 써 온 wagmi 직접 읽기. 두 경로는 함께 쓴다.

세 프로그램을 잇는 연결 고리는 각각 하나씩이에요.

연결	무엇으로?	핵심 파일
솔리디티 → 서브그래프	**이벤트**	`AMM.sol`의 `emit` ↔ `subgraph.yaml`의 `event:` ↔ `amm.ts`의 handler
서브그래프 내부	**엔티티** (테이블)	`schema.graphql` ↔ `amm.ts`의 `new` / `load` / `save`
서브그래프 → 프론트엔드	**GraphQL** 조회	서브그래프 URL ↔ `useQuery` + `fetch`

2 솔리디티 ↔ amm.ts: 이벤트로 연결돼요

서브그래프는 컨트랙트 함수를 직접 보지 않아요. **`emit` 된 이벤트만** 봐요. 그래서 AMM.sol의 `emit` 6개와 amm.ts의 handler 6개가 하나씩 짝을 이루고, 그 사이를 `subgraph.yaml`이 이어 줘요.

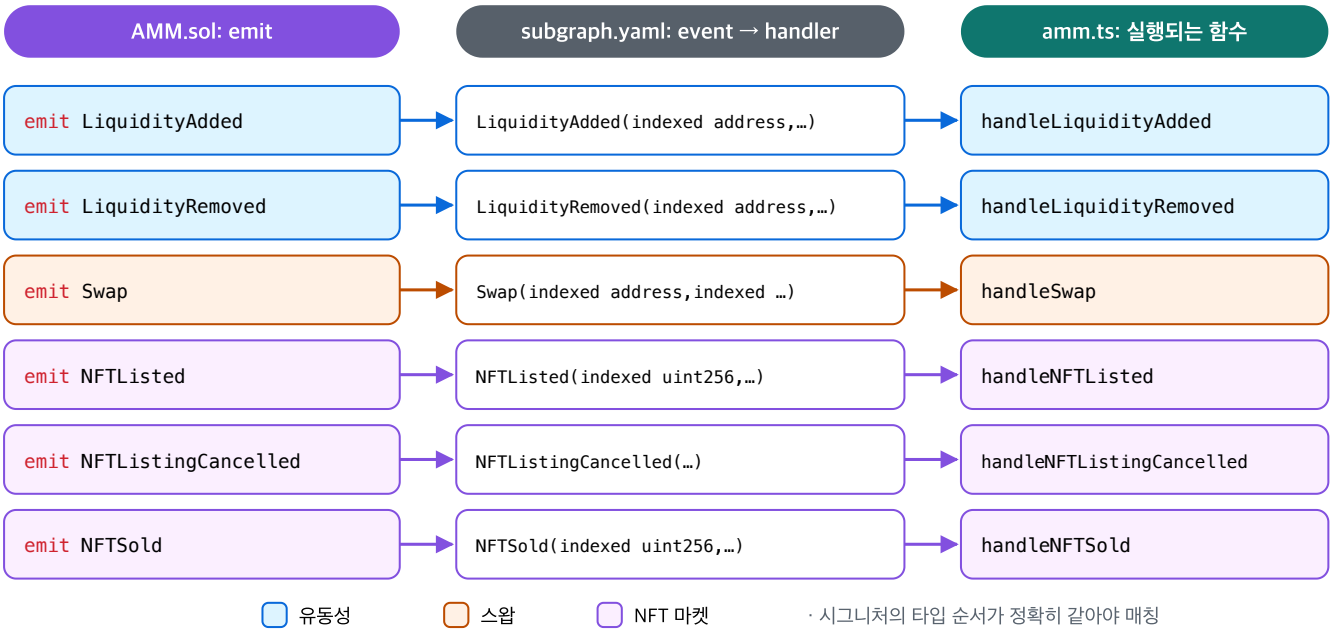


그림 2. emit 하나 = handler 하나. subgraph.yaml이 이벤트 시그니처를 보고 어느 함수를 실행할지 정한다.

`event.params.amountIn`은 어디서 생길까? `graph codegen`

amm.ts에서 쓰는 `LiquidityAdded`, `Pool` 같은 클래스는 직접 만든 게 아니라 **자동 생성**된 거예요. 입력은 두 가지예요.

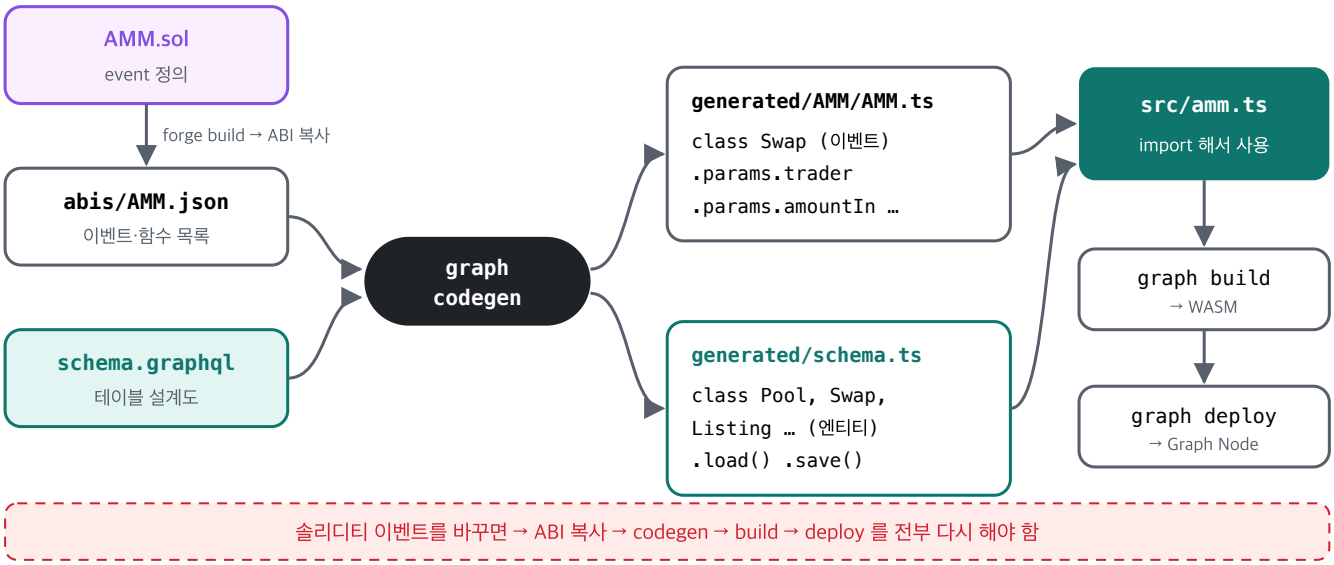


그림 3. ABI와 스키마가 입력, `generated/` 아래 파일이 출력. amm.ts는 생성된 클래스를 import해서 쓴다.

예를 들어 솔리디티 이벤트가 이렇게 생겼으면,

AMM.sol

```
event LiquidityRemoved(
  address indexed provider, // [0]
  uint256 liquidity,          // [1]
  uint256 amountA,            // [2]
  uint256 amountB             // [3]
);
```

generated/AMM/AMM.ts (자동 생성)

```
get provider()  { return parameters[0] }
get liquidity() { return parameters[1] }
get amountA()   { return parameters[2] }
get amountB()   { return parameters[3] }
```

솔리디티에서 붙인 **이름과 순서가 그대로** 게터가 돼요. 그래서 amm.ts에서는 `event.params.amountA`처럼 이름으로 꺼내 쓸 수 있어요.

3 스왑 한 번을 끝까지 따라가 보기

사용자가 SW42(토큰 A) 100개를 넣고 R2B(토큰 B) 90개를 받았다고 해 볼게요. (숫자는 wei가 아니라 단순화한 값이에요.)

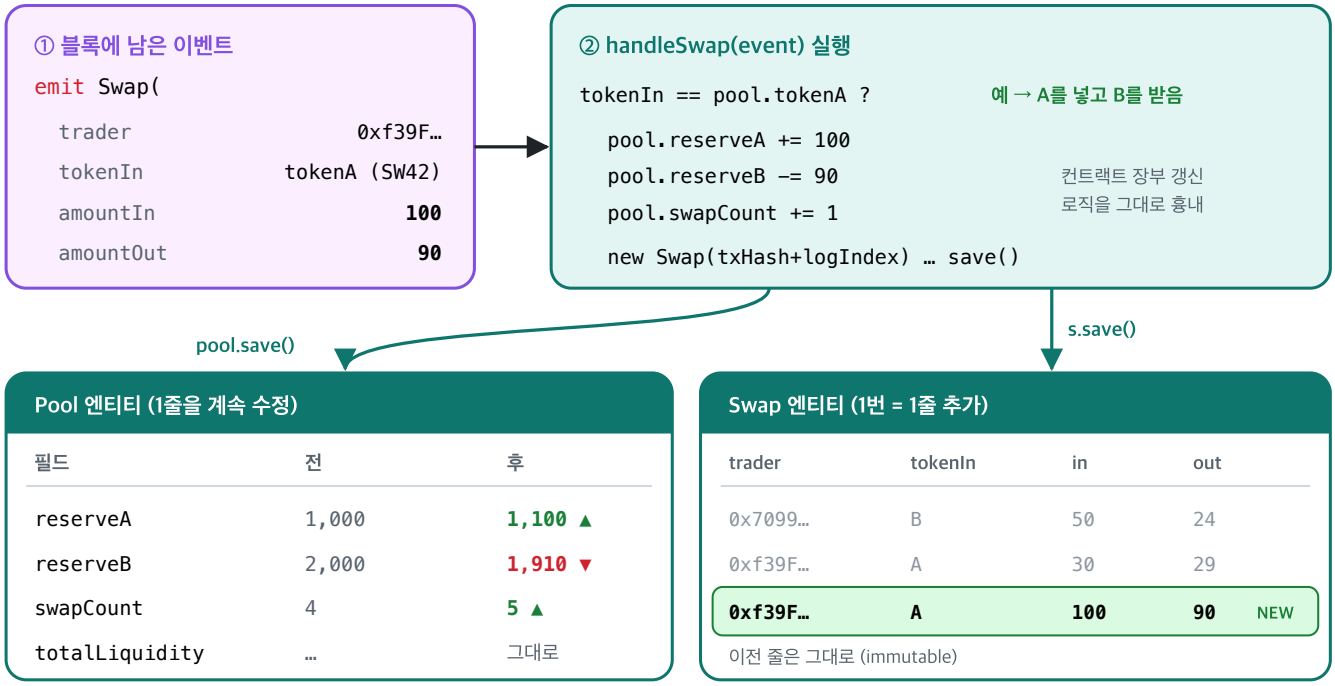


그림 4. 이벤트 하나로 두 테이블이 바뀐다. Pool은 "현재 상태"를 수정하고, Swap은 "기록"을 한 줄 추가한다.

handleSwap의 리저브 계산은 AMM.sol `swap()`의 "5) 장부 갱신"을 그대로 옮긴 거예요.

AMM.sol

```
if (isAToB) {
  reserveA += amountIn;
  reserveB -= amountOut;
} else {
  reserveB += amountIn;
  reserveA -= amountOut;
}
```

amm.ts

```
if (tokenIn.equals(pool.tokenA)) {
  reserveA = reserveA.plus(amountIn)
  reserveB = reserveB.minus(amountOut)
} else {
  reserveB = reserveB.plus(amountIn)
  reserveA = reserveA.minus(amountOut)
}
```

> 이벤트에는 `tokenIn`만 있고 "A→B인지"는 없어요. 그래서 `getPool`이 Pool을 처음 만들 때 `AMM.bind(address).tokenA()`로 **컨트랙트의 view 함수를 한 번 호출해서** 토큰 주소를 저장해 둬요(`[1]`). 그 뒤로는 그 값과 비교해서 방향을 정해요.

4 엔티티: "기록"과 "현재 상태"

`schema.graphql`의 `type … @entity` 하나가 DB 테이블 하나예요. 성격이 두 가지로 나뉘어요.

종류	엔티티	amm.ts에서 하는 일
`immutable: true` 기록 (로그)	`Swap`, `LiquidityEvent`	`new` → `save()`만. 한 번 쓰면 다시 안 건드림. id = tx 해시 + 로그 번호 (한 tx에 이벤트가 여러 개일 수 있어서)
`immutable: false` 현재 상태	`Pool`, `Listing`, `Nft`	`load()` → 수정 → `save()`. 같은 id의 줄을 계속 덮어씀.

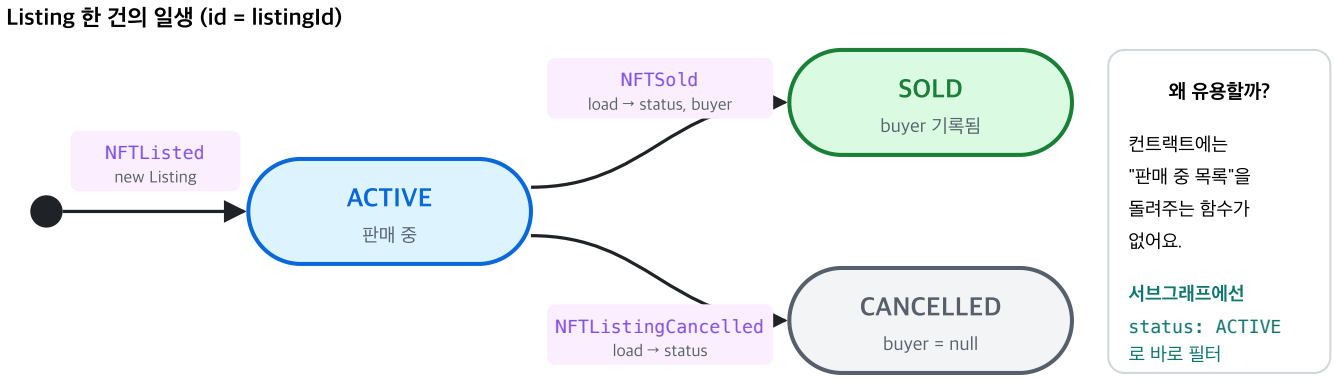


그림 5. 같은 id의 Listing 한 줄이 이벤트에 따라 상태만 바뀐다. 끝난 판매도 지워지지 않고 기록으로 남는다.

5 서브그래프 ↔ 프론트엔드: GraphQL로 연결돼요

배포된 서브그래프는 **GraphQL API(URL 하나)**를 열어 줘요. 프론트엔드는 거기에 "어떤 테이블에서, 어떤 조건으로, 어떤 필드를" 달라고 POST 요청을 보내요.

요청 (GraphQL)

```
{
  listings(
    where: { status: ACTIVE }
    orderBy: createdAt
    orderDirection: desc
  ) {
    id
    tokenId
    price
    paymentToken
    seller
  }
}
```

응답 (JSON)

```
{ "data": {
    "listings": [
      { "id": "3",
        "tokenId": "7",
        "price": "5000000000000000000",
        "paymentToken": "0x5FbD…",
        "seller": "0xf39F…" },
      …
    ]
} }
```

프론트에서는 앞에서 배운 `useQuery`를 그대로 써요. NFT 메타데이터를 가져올 때처럼, 서브그래프도 블록체인 밖의 **일반 HTTP API**이기 때문이에요.

```
const listings = useQuery({
  queryKey: ['listings'],
  queryFn: () =>
    fetch(SUBGRAPH_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        query: `{ listings(where: { status: ACTIVE }) { id tokenId price } }`,
      }),
    }).then((r) => r.json()),
})
// BigInt 필드는 문자열로 와요 → BigInt(l.price)로 바꿔서 formatUnits에 넘김
```

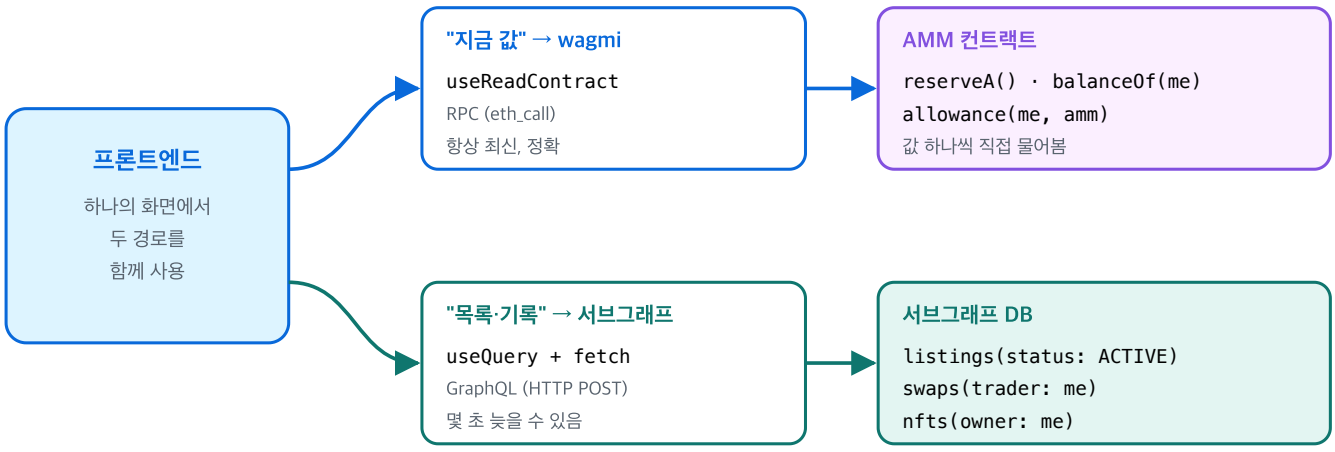
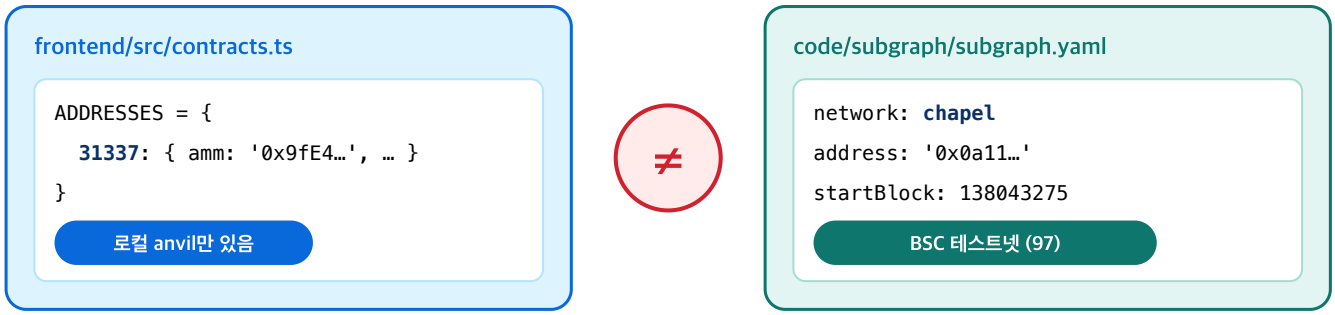


그림 6. 두 경로는 경쟁 관계가 아니라 역할 분담이다. 하나의 값은 wagmi로, 여러 줄의 목록과 과거 기록은 서브그래프로.

알고 싶은 것	wagmi로 직접 읽기	서브그래프
지금 리저브	✓ `reserveA()` 한 번	가능 (인덱싱 지연이 조금 있음)
판매 중인 NFT 전부	✗ 목록 함수가 없어서 id를 하나씩 조회해야 함	✓ `listings(where: {status: ACTIVE})`
내 스왑 기록	✗ 컨트랙트는 과거 기록을 저장하지 않음	✓ `swaps(where: {trader: me})`
내 NFT	△ 지금은 `ownerOf`를 0~19까지 20번 호출	✓ `nfts(where: {owner: me})` 한 번
정렬 · 필터 · 페이지	✗	✓ `orderBy`, `where`, `first`, `skip`

6 주의: 지금은 서로 다른 체인을 보고 있어요



anvil에 연결한 화면에 BSC 테스트넷 기록이 섞이면 숫자가 맞지 않아요

그림 7. 프론트엔드는 로컬 체인, 서브그래프는 BSC 테스트넷을 본다.

나중에 둘을 연결할 때는 이렇게 하면 돼요.

1. `ADDRESSES`에 `97: { amm: '0x0a11…', nft: '0x7DA2…', … }`를 추가해요 (`deployment/97.json`에서 복사).
2. 서브그래프 조회는 체인이 97일 때만 켜요: `enabled: connection.chainId === 97`

7 정리: 파일별 역할

파일	역할	누구와 연결?
`code/src/AMM.sol`	이벤트를 `emit`	→ 블록체인 로그 → 서브그래프
`subgraph/abis/AMM.json`	컨트랙트 ABI 사본	AMM.sol을 빌드한 결과 → codegen 입력
`subgraph/subgraph.yaml`	어느 체인 · 주소 · 이벤트를 어느 handler로	AMM.sol의 이벤트 ↔ amm.ts의 함수
`subgraph/schema.graphql`	DB 테이블 설계도	→ codegen → amm.ts가 저장 / 프론트가 조회
`subgraph/src/amm.ts`	이벤트 → 테이블 변환 로직	입력은 이벤트, 출력은 엔티티
`frontend/src/App.tsx`	화면	지금 값은 wagmi로, 목록과 기록은 GraphQL로